

权重的初始值 与 梯度消失

1. Sigmoid 激活函数

1.1 函数与导数

Sigmoid 函数是一种常用的激活函数，但在反向传播的过程中，容易引起梯度消失的问题，即随着神经网络层数的加深，反向传播时传回的梯度值会逐渐减小。

Sigmoid 激活函数的数学表达式为：

$$f(x) = \frac{1}{1 + \exp(-x)}$$

我们对其求导，得到导函数：

$$f'(x) = f(x) \times (1 - f(x))$$

1.2 Python 实现

我们用 Python 代码实现这个类：

```
class Sigmoid:
    def __init__(self):
        self.out = None

    def forward(self, x):
        out = 1 / (1 + np.exp(-x))
        self.out = out
        return out

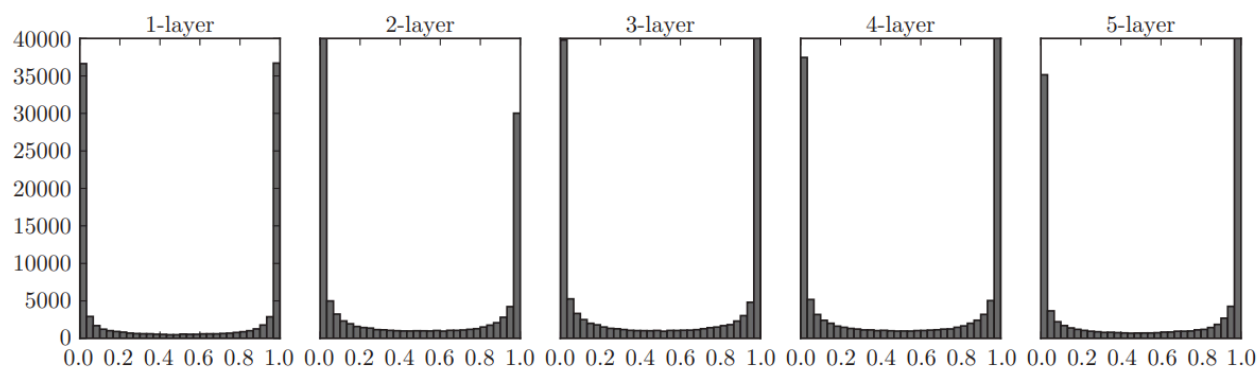
    def backward(self, dout):
        # dx = dout * (self.out ** 2) * np.exp(-x) 与下方代码仅在数学表达上完全
        # 相同
        dx = dout * self.out * (1 - self.out)
        return dx
```

我们可以计算出导函数的最大值在 $f(x) = 0.5$ 出现，为 0.25，因此，在层数较多的神经网络中，就容易出现反向传播后的梯度极小的情况。而如果输入值的绝对值过大，Sigmoid 的输出值会接近 0 或 1，这时情况会更极端，其导函数值将接近于 0，进而使得反向传播回去的梯度值趋近于 0，这样一来，靠前的层级学习效果将会很差，因为梯度过小，对参数的影响微乎其微。

1.3 不同权重初始值下的激活值分布

标准差为 1 的高斯分布

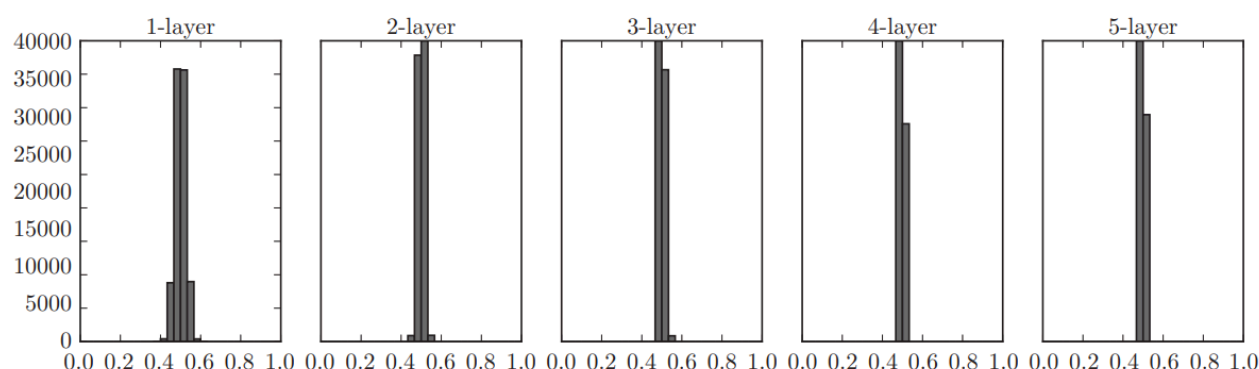
标准差为 1 的高斯分布作为权重初始值时的各层激活值的分布：



这里就能很明显的看出来，偏向 0 或 1 的数据使得反向传播的梯度值快速减小，梯度消失明显。

标准差为 0.01 的高斯分布

标准差为 0.01 的高斯分布作为权重初始值时的各层激活值的分布：



此时激活值的分布有所偏向，即大部分神经元的输出结果差异很小，表现力受限。

2. ReLU 激活函数

2.1 函数定义

ReLU 函数也是一种常用的激活函数，其数学表达式为：

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

从表达式中可以看出，ReLU 能缓解梯度消失的问题，因为当 $x > 0$ 时，其导数值为 1，梯度会近似原样通过，但问题出现在 $x \leq 0$ 的情况，若某个神经元长期落在负数区域，就会导致传回的梯度值为 0，即神经元死亡，简单来说就是不再学习，参数不再更新了。

Leaky ReLU

可以通过换用 Leaky ReLU 激活函数来缓解这个问题：

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

2.2 Python 实现

我们用 Python 代码实现这个类：

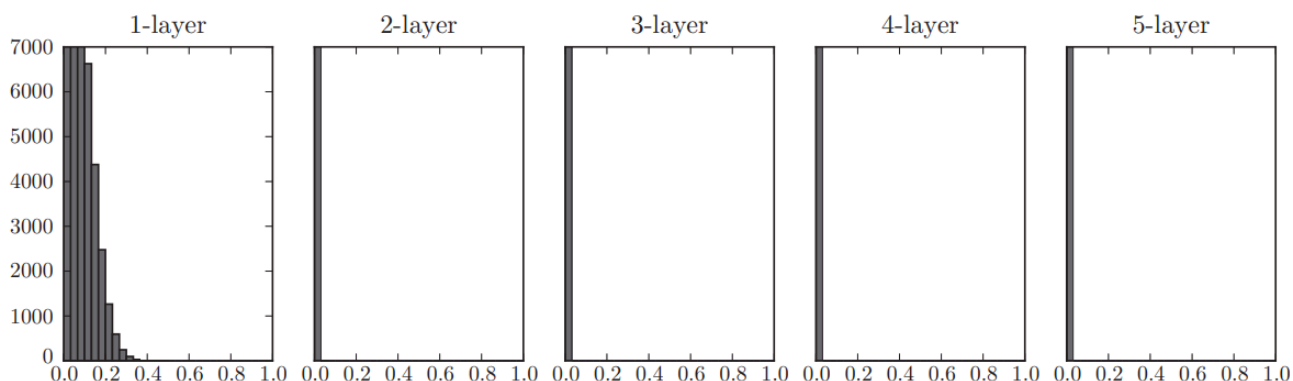
```
class ReLU:
    def __init__(self):
        self.mask = None

    def forward(self, x):
        self.mask = (x <= 0) # 找出数组中 x 小于等于 0 的位置
        out = x.copy()
        out[self.mask] = 0 # 把逻辑为 True 的位置全部赋值为 0
        return out

    def backward(self, dout):
        dout[self.mask] = 0
        dx = dout * 1 # ReLU 函数中 x 大于 0 时取值为 x，即导数值为 1
        return dx
```

2.3 标准差为 0.01 的高斯分布

标准差为 0.01 的高斯分布作为权重初始值时的各层激活值的分布：



不难看出，各层的激活值很小，此时神经网络上传递的是非常小的值，说明逆向传播时权重的梯度也同样很小，说明实际上学习非常缓慢。

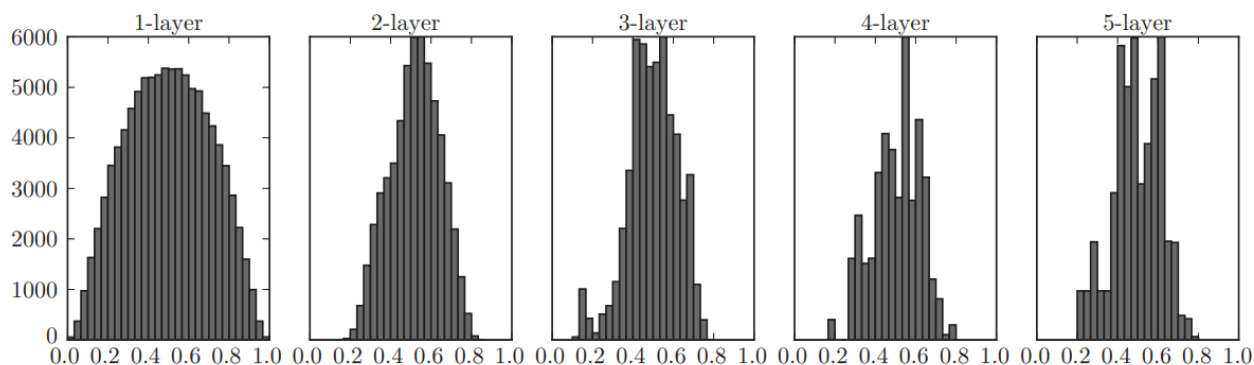
3. 调整权重初始值

3.1 Xavier 初始值

我们可以通过调整权重的初始值来缓解梯度消失，对于 Sigmoid 激活函数，可以用 Xavier 初始值，即使用标准差为 $1/\sqrt{n}$ （ n 为前一层的节点数）的高斯分布初始化权重，代码表示为：

```
w = np.random.randn(node_num, node_num) / np.sqrt(node_num)
```

此时各层激活值的分布如下：



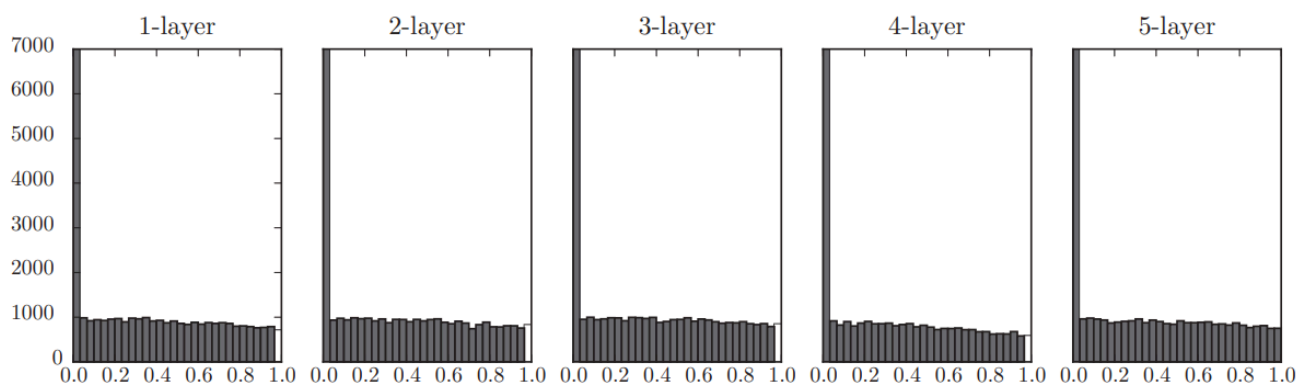
可以看出，相较于之前，图像呈现出了更有广度的分布，所以 Sigmoid 函数的表现力不受限制，有了高效学习的可能。

3.2 He 初始值

而对于 ReLU 激活函数，通常更适合 He 初始化，即使用标准差为 $\sqrt{2/n}$ (n 为前一层的节点数) 的高斯分布作为权重初始值，代码表示为：

```
w = np.random.randn(node_num, node_num) * np.sqrt(2 / node_num)
```

此时各层激活值的分布如下：



从图中看出，当初始值为 He 初始值时，各层中分布的广度相同。由于即便层加深，数据的广度也能保持不变，因此逆向传播时，也会传递合适的值。

🔗 He 初始值的直观理解

简单理解一下为什么 He 初始值是标准差为 $\sqrt{2/n}$ (n 为前一层的节点数) 的高斯分布：模型训练时，我们的主要目的并不是保证每个数不变，而是保证信号整体的数值尺度稳定。因此，我们其实需要保证的是各层传出来的数值方差尽量不变。对比 Sigmoid 激活函数，ReLU 有一个明显的特点，即 $x < 0$ 时信号会被截断，输出值为 0。当输入在零附

近近似对称、正负值比例接近一半时，大约一半的信号被截成 0 了，而为了弥补这部分的损失，我们扩大权重方差 2 倍，保证输出信号整体稳定。